

Informe Técnico Integrado

Entrenamiento, Despliegue y Monitoreo del Modelo de Predicción de Abandono de Clientes (Churn)

Cliente: AndesLink Servicios Digitales S.A.

Autor: Facundo Zubeldia

Julio 2026

1. Problema de Negocio y Objetivo del Proyecto

AndesLink Servicios Digitales S.A. comercializa planes de suscripción mensual para servicios digitales orientados a consumidores finales. Ante una tasa creciente de cancelación voluntaria de clientes (churn), que impacta los ingresos recurrentes y el costo de adquisición de usuarios, se planteó el desarrollo de un modelo de clasificación binaria capaz de estimar la probabilidad de abandono de cada cliente, a partir de variables de comportamiento, antigüedad, facturación y relación con el servicio.

El proyecto se desarrolló en tres fases: una primera fase centrada en el entrenamiento del modelo, una segunda enfocada en su despliegue como servicio productivo mediante una API REST, un dashboard de consumo y contenedores Docker, y una tercera dedicada a la observabilidad del sistema (monitoreo técnico y de calidad de datos). Este documento integra las tres fases, incorporando las mejoras aplicadas a partir de las revisiones técnicas realizadas en cada etapa.

2. Dataset

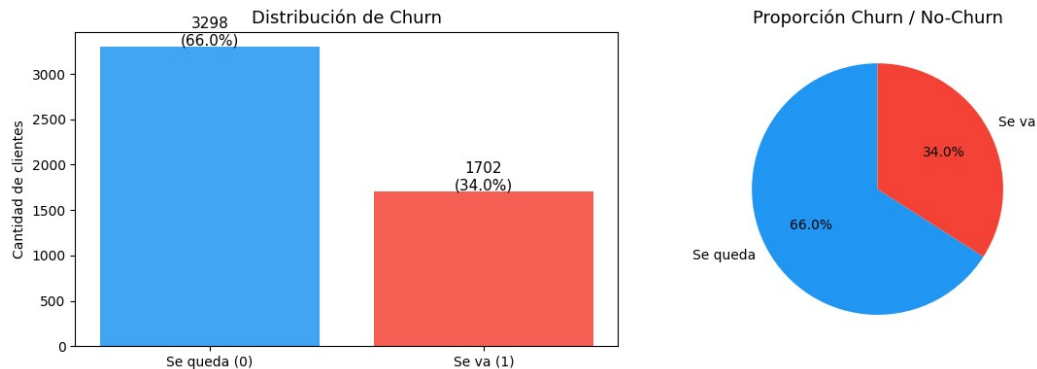
Se utilizó un dataset sintético de 5.000 registros y 16 columnas (15 features + variable objetivo churn), diseñado para simular el comportamiento de clientes de una empresa de suscripciones digitales. El dataset está versionado con DVC; adicionalmente, tras la revisión de la Fase 1, se incorporó el archivo CSV directamente al repositorio para asegurar portabilidad ante la limitación del remote local original.

Variables principales: antigüedad (tenure_months), cargo mensual y total, tickets de soporte, pagos tardíos, uso promedio mensual, tipo de contrato, método de pago, tipo de servicio de internet, región, streaming, paquete de seguridad, cantidad de productos, edad y si ingresó con promoción.

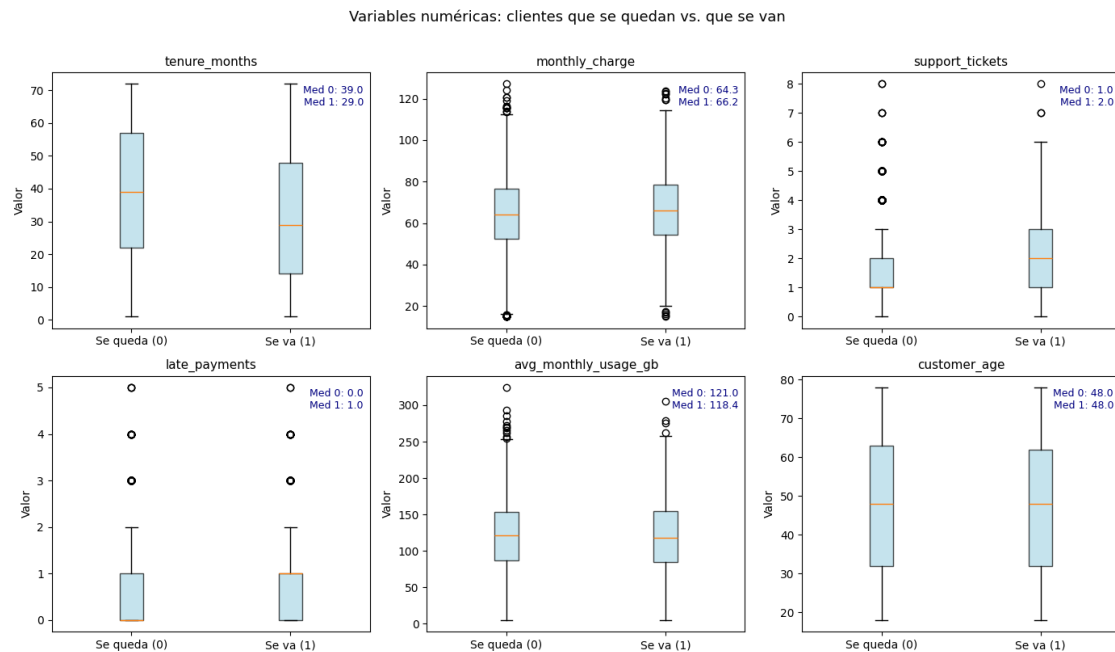
Limitaciones: al ser sintético, los patrones pueden no reflejar comportamiento real; total_charges es una variable derivada con alta colinealidad ($r=0.87$) respecto a tenure_months; las correlaciones lineales con churn son bajas (máx. $r=-0.17$), lo que sugiere relaciones no lineales.

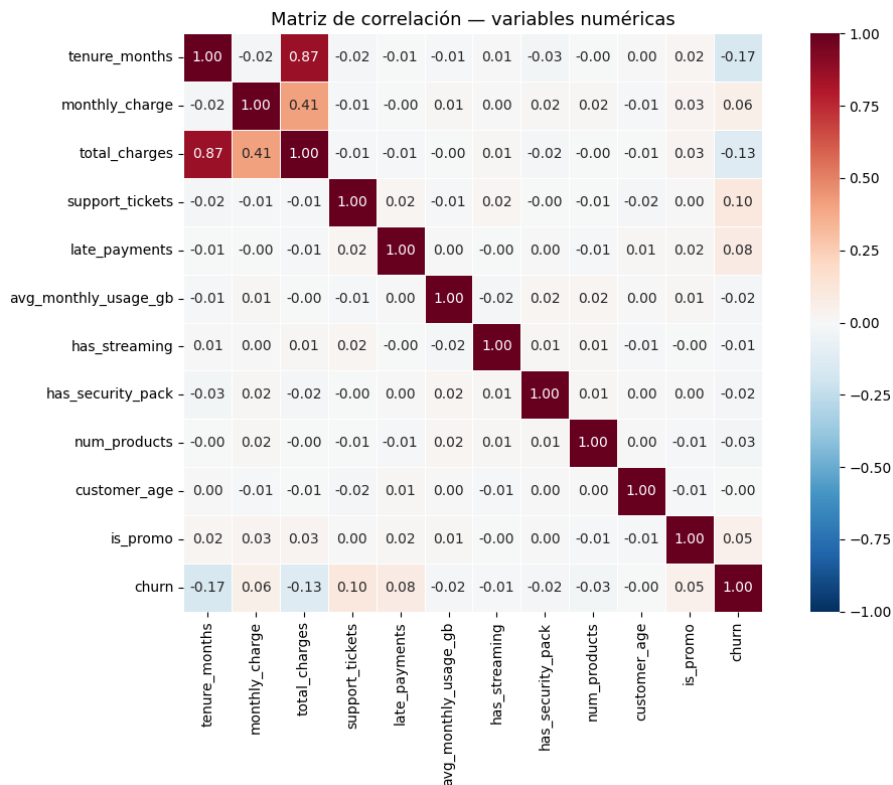
3. Análisis Exploratorio (EDA)

El dataset presenta un desbalance moderado: 66% de clientes que permanecen (clase 0) vs. 34% que abandonan (clase 1), lo que descarta el accuracy como métrica principal.



Las variables numéricas con mayor separación entre grupos son `tenure_months` (mediana 39 vs. 29 meses) y `support_tickets` (mediana 1 vs. 2), indicando que clientes nuevos con más tickets de soporte tienen mayor propensión al churn.

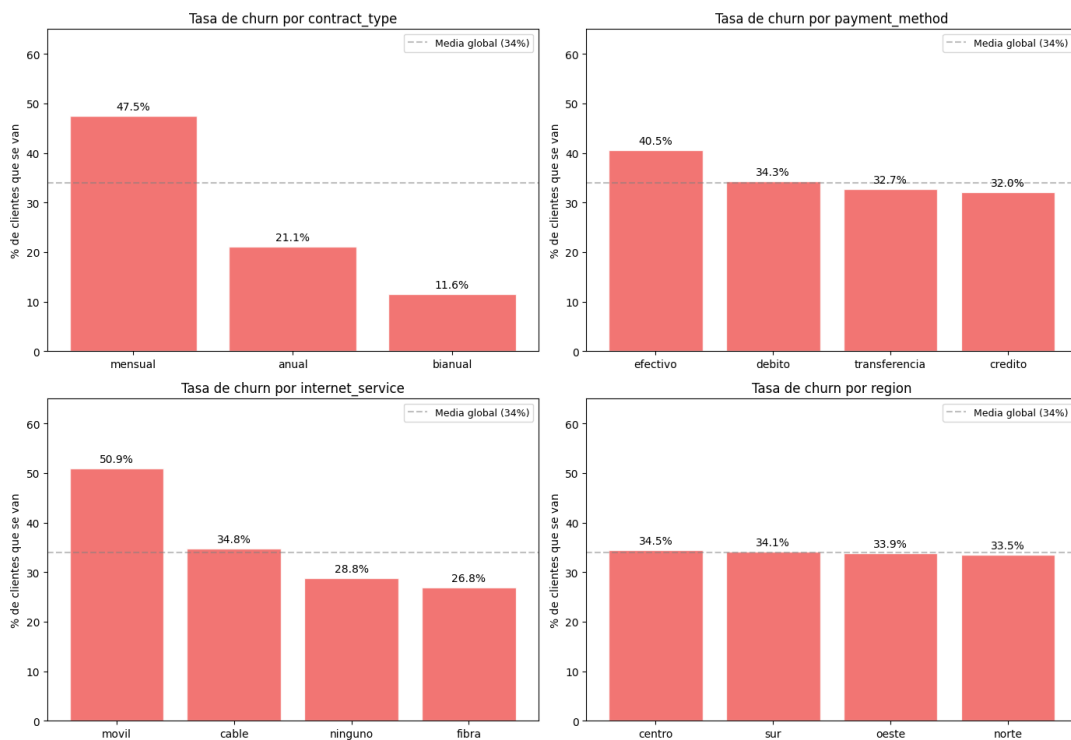




Hallazgos por variable categórica

- **contract_type:** contratos mensuales presentan 47,5% de churn vs. 11,6% en bianuales.
- **internet_service:** el servicio móvil presenta 50,9% de churn; fibra solo 26,8%.
- **payment_method:** el pago en efectivo presenta mayor churn (40,5%), asociado a menor automatización.
- **region:** sin diferencias significativas (33–34%); el churn no tiene componente geográfico.

Tasa de churn por variable categórica



4. Preparación de Datos y Entrenamiento

Se eliminó el identificador `customer_id` y se verificó la ausencia de duplicados y valores nulos. Se aplicó un Pipeline de scikit-learn con `StandardScaler` para variables numéricas y `OneHotEncoder` (`handle_unknown='ignore'`) para variables categóricas, serializado junto con el clasificador en un único artefacto `.pkl`.

Tras la revisión de la Fase 1, se incorporó split estratificado (`stratify=y`) en la separación train/test (80/20), garantizando que el desbalance 66/34 se mantenga en ambos subconjuntos.

Comparación de modelos

```
=== COMPARACIÓN FINAL ===
                Accuracy  F1 (churn)  Recall (churn)  ROC-AUC
Modelo
Logistic Regression    0.672    0.6019    0.7294    0.7580
Decision_Tree          0.635    0.5701    0.7118    0.7083
Random_Forest          0.707    0.4832    0.4029    0.7256

→ Modelo seleccionado: el de mayor F1 (churn)
Ganador: Logistic_Regression
```

Se seleccionó Logistic Regression con `class_weight='balanced'` por presentar el mejor F1-score y Recall de la clase churn, la métrica de mayor relevancia de negocio dado que un recall bajo implica campañas de retención ineficaces. Random Forest, pese a tener el mayor accuracy, obtuvo el peor F1 y Recall, confirmando que el accuracy no es una métrica adecuada frente al desbalance de clases.

Tracking de experimentos

Todas las corridas se registraron con MLflow bajo el experimento 'AndesLink_Churn_Prediction', logueando F1, Recall y ROC-AUC de cada modelo.

☐	Run Name	Created ↕	Dataset	Duration	Source	Models
☐	Random_Forest	✓ 2 hours ago	-	432ms	01_eda...	-
☐	Decision_Tree	✓ 2 hours ago	-	34ms	01_eda...	-
☐	Logistic_Regression	✓ 2 hours ago	-	35ms	01_eda...	-
☐	charming-shrew-580	✓ 2 hours ago	-	0.9s	01_eda...	-

5. Arquitectura de Despliegue

El modelo entrenado se expone como servicio productivo mediante dos componentes orquestados con Docker Compose, comunicados a través de la red interna de Docker:

Servicio	Tecnología	Puerto	Función
API de inferencia	FastAPI + Uvicorn	8000	Recibe datos JSON y devuelve la predicción
Dashboard GUI	Streamlit	8501	Interfaz visual para usuarios finales

Flujo de inferencia: el usuario ingresa datos del cliente en el dashboard; Streamlit envía un POST a la API con los datos en JSON; la API aplica el pipeline serializado (preprocesamiento + modelo) y

responde con probabilidad de churn, predicción binaria y nivel de riesgo; el dashboard muestra el resultado con métricas e indicadores visuales.

La comunicación entre contenedores se resuelve mediante el nombre del servicio (`http://api:8000`) en lugar de `localhost`, configurado a través de la variable de entorno `API_URL` sin necesidad de modificar código.

6. API de Inferencia (FastAPI)

La API expone un endpoint de verificación de estado y el endpoint principal de predicción. A partir de las observaciones de la revisión de la Fase 2, el chequeo de salud se reforzó para validar explícitamente que el modelo esté cargado y disponible para inferir, y no solo que la aplicación haya iniciado.

Endpoint	Método	Descripción
/health	GET	Verifica que el servicio y el modelo estén operativos
/predict	POST	Recibe datos de un cliente y devuelve la predicción

Contrato de entrada (`ClienteInput`): 15 campos obligatorios validados con Pydantic (tipos y rangos). Las variables categóricas se restringen mediante `Literal/Enum` para limitar los valores aceptados a los dominios válidos, corrigiendo la observación de la revisión sobre validaciones no formales.

Contrato de salida (`PrediccionOutput`): `churn` (0/1), `probabilidad_churn` (float entre 0.0 y 1.0) y `riesgo` ('bajo' <0.4, 'medio' 0.4–0.7, 'alto' ≥0.7).

AndesLink Churn Predictor API 1.0.0 OAS 3.1

api.openapi.json

API de inferencia para predicción de abandono de clientes.

default ^

- GET /metrics Metrics
- GET / Root
- GET /health Health Check
- POST /predict Predecir

Schemas ^

- `ClienteInput` > Expand all object
- `HTTPValidationError` > Expand all object
- `PrediccionOutput` > Expand all object
- `ValidationError` > Expand all object

7. Dashboard de Predicción (Streamlit)

El dashboard permite a usuarios no técnicos ingresar los datos de un cliente y obtener su probabilidad de churn de forma inmediata, consumiendo la API internamente. Incluye formulario con sliders y selectores para los 15 campos del modelo, indicador visual de riesgo por zonas, tabla resumen del cliente analizado y manejo de errores de conexión.



8. Contenedorización con Docker

Ambos servicios están empaquetados en imágenes independientes orquestadas con Docker Compose: Dockerfile para la API (basada en python:3.11-slim), Dockerfile.streamlit para el dashboard, y requirements-docker.txt con las dependencias mínimas de producción.

Se corrigió el healthcheck de Docker Compose: en lugar de depender de curl (no incluido por defecto en la imagen slim), se implementó una verificación basada en Python contra el endpoint /health, y se configuró depends_on con condición de servicio saludable (service_healthy) para asegurar que el dashboard no arranque hasta que la API esté efectivamente operativa.

- Levantar los servicios: `docker-compose up --build -d`
- Verificar estado: `docker-compose ps`
- Detener servicios: `docker-compose down`

9. Tests Automatizados (pytest)

Se implementaron tests automatizados con TestClient de FastAPI, ejecutados también en GitHub Actions con cada push, junto con un workflow que valida la construcción de ambas imágenes Docker.

1. **test_root** — GET / responde 200 con status: ok y devuelve info del modelo.
2. **test_health_check** — GET /health confirma que el modelo está cargado (modelo: cargado) y puede inferir (inferencia: ok).
3. **test_prediccion_cliente_valido** — POST /predict con un cliente válido devuelve 200 y los campos churn, probabilidad_churn (0–1) y riesgo (bajo/medio/alto) bien formados.
4. **test_prediccion_payload_incompleto** — un payload con solo tenure_months (faltan campos obligatorios) devuelve 422.
5. **test_prediccion_tipo_incorrecto** — tenure_months como string ("doce") en vez de número devuelve 422.
6. **test_categoria_invalida_contract_type** — contract_type: "trimestral" (fuera del dominio permitido) devuelve 422.
7. **test_categoria_invalida_internet_service** — internet_service: "satelital" (fuera del dominio permitido) devuelve 422.
8. **test_formato_respuesta** — la respuesta tiene exactamente las claves churn, probabilidad_churn, riesgo, con los tipos correctos (int, float, str).
9. **test_prediccion_cliente_bajo_riesgo** — un cliente "fiel" (bianual, fibra, sin tickets/pagos tardíos) debe dar probabilidad de churn < 0.6.
10. **test_prediccion_cliente_alto_riesgo** — un cliente "en riesgo" (mensual, móvil, muchos tickets) debe dar probabilidad de churn > 0.2.
11. **test_payload_vacio** — un POST /predict con {} devuelve 422.

```
(venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ pytest tests/ -v
tests/test_api.py::test_prediccion_cliente_valido PASSED
tests/test_api.py::test_prediccion_payload_incompleto PASSED
tests/test_api.py::test_prediccion_tipo_incorrecto PASSED
tests/test_api.py::test_categoria_invalida_contract_type PASSED
tests/test_api.py::test_categoria_invalida_internet_service PASSED
tests/test_api.py::test_formato_respuesta PASSED
tests/test_api.py::test_prediccion_cliente_bajo_riesgo PASSED
tests/test_api.py::test_prediccion_cliente_alto_riesgo PASSED
tests/test_api.py::test_payload_vacio PASSED

===== 11 passed in 1.39s =====
(venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$
```

10. Mejoras Implementadas tras las Revisiones de las Fases 1 y 2

De la Fase 1 (Entrenamiento)

- Se incorporó split estratificado (stratify=y) para preservar la proporción 66/34 en train y test.
- Se resolvió la limitación de portabilidad de DVC incluyendo el dataset directamente en el repositorio (en un entorno productivo el dataset residiría en almacenamiento en la nube; por cuestiones de presupuesto y tamaño del dataset se decidió mantenerlo en el repositorio).

De la Fase 2 (Despliegue)

- Se agregó un endpoint /health dedicado que valida explícitamente que el modelo esté cargado.
- Se corrigió el healthcheck de Docker Compose, reemplazando la dependencia de curl por una verificación en Python.

- Se configuró `depends_on` con condición `service_healthy` para ordenar correctamente el arranque de los servicios.
- Se restringieron las variables categóricas del contrato de entrada mediante `Literal/Enum`.
- Se centralizó la configuración de rutas, `thresholds` de riesgo y versión del modelo.

11. Conclusiones de las Fases 1 y 2

El proyecto cubre el ciclo completo de un flujo de MLOps: desde la exploración y el entrenamiento del modelo hasta su despliegue como servicio productivo con API, dashboard, contenedores y pruebas automatizadas. Las correcciones aplicadas tras las revisiones de las fases 1 y 2 mejoraron tanto la calidad del modelo (`recall`) como la robustez operativa del despliegue (`healthcheck`, validaciones, portabilidad).

Próximos pasos: Fase 3

- Instrumentar métricas técnicas con Prometheus (`requests`, latencia, errores).
- Visualizar métricas operacionales en Grafana.
- Implementar monitoreo de drift con Evidently.

12. Fase 3: Observabilidad del Sistema

En esta tercera y última fase se implementa la capa de observabilidad del sistema MLOps. El objetivo es monitorear el comportamiento técnico del servicio en producción (latencia, tráfico, errores) y la calidad de los datos que recibe el modelo (drift), para detectar degradación del rendimiento antes de que impacte en el negocio.

13. Arquitectura Completa del Sistema

El sistema final integra 4 servicios orquestados con Docker Compose:

Servicio	Tecnología	Puerto	Función
API de inferencia	FastAPI + Uvicorn	8000	Predicción de churn + métricas
Dashboard GUI	Streamlit	8501	Interfaz para usuarios finales
Métricas	Prometheus	9091	Recolección de métricas técnicas
Visualización	Grafana	3001	Dashboard de monitoreo operativo

Flujo de monitoreo

1. La API expone métricas en `/metrics` (formato Prometheus) gracias a `prometheus-fastapi-instrumentator`.
2. Prometheus hace scraping de ese endpoint cada 15 segundos.
3. Grafana consulta Prometheus y visualiza las métricas en tiempo real.
4. Evidently corre como script separado y genera un reporte HTML de drift comparando datos de referencia vs producción.


```

• facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ source /home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos/venv/bin/activate
• (venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ docker-compose ps
NAME                IMAGE              COMMAND                  SERVICE    CREATED   STATUS    PORTS
andeslink-api        lab-mineria-de-datos-api  "uvicorn src.api:app..."  api        42 minutes ago  Up 42 minutes (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/
tcp
andeslink-dashboard  lab-mineria-de-datos-dashboard  "streamlit run src/a..."  dashboard  42 minutes ago  Up 42 minutes             0.0.0.0:8501->8501/tcp, [::]:8501->8501/
tcp
andeslink-grafana     grafana/grafana:latest  "/run.sh"                grafana     42 minutes ago  Up 42 minutes             0.0.0.0:3001->3000/tcp, [::]:3001->3000/
tcp
andeslink-prometheus prom/prometheus:latest  "/bin/prometheus --c..."  prometheus  42 minutes ago  Up 42 minutes             0.0.0.0:9091->9090/tcp, [::]:9091->9090/
tcp
• (venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$

```

Servicios del sistema en ejecución: API (healthy), dashboard, Prometheus y Grafana.

14. Mejoras Incorporadas respecto a la Fase 2

14.1 Endpoint /health real

Se reemplazó el health check básico por un endpoint que valida que el modelo está cargado y puede realizar una predicción de prueba. El healthcheck de Docker apunta a este endpoint, garantizando que el dashboard solo arranque cuando la API está realmente operativa.

14.2 Validación estricta de categorías

Se reemplazó str por Literal de Python para las variables categóricas (contract_type, payment_method, internet_service, region). Ahora la API devuelve error 422 automáticamente si se envía un valor fuera del dominio permitido, sin necesidad de código adicional.

14.3 Logging estructurado

Se agregó logging con formato timestamp para registrar cada predicción realizada, incluyendo probabilidad de churn y nivel de riesgo. Esto permite auditoría de las inferencias en producción.

14.4 Tests ampliados a 11

Se agregaron tests de categorías inválidas, endpoint /health, payload vacío y umbrales de negocio más exigentes (< 0.5 y > 0.5 en vez de umbrales triviales).

15. Monitoreo Técnico con Prometheus y Grafana

15.1 Métricas recolectadas

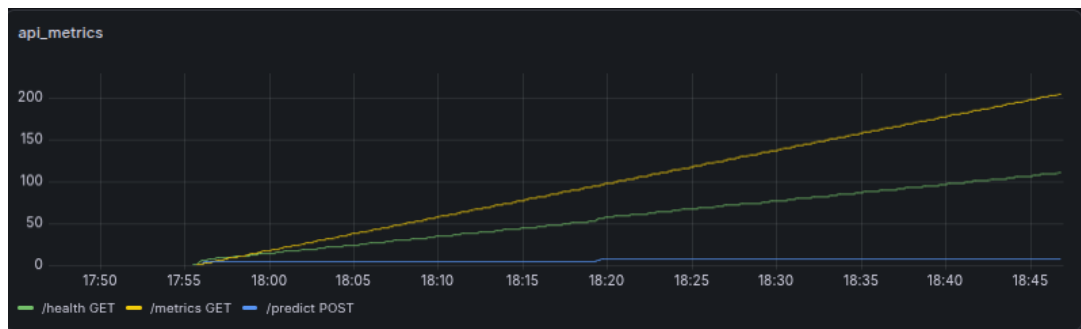
La librería prometheus-fastapi-instrumentator instrumenta automáticamente la API y expone las siguientes métricas en /metrics:

- http_requests_total — contador de requests por endpoint y método HTTP
- http_request_duration_seconds — histograma de latencia por endpoint
- process_cpu_seconds_total — uso de CPU del proceso
- process_resident_memory_bytes — uso de memoria RAM

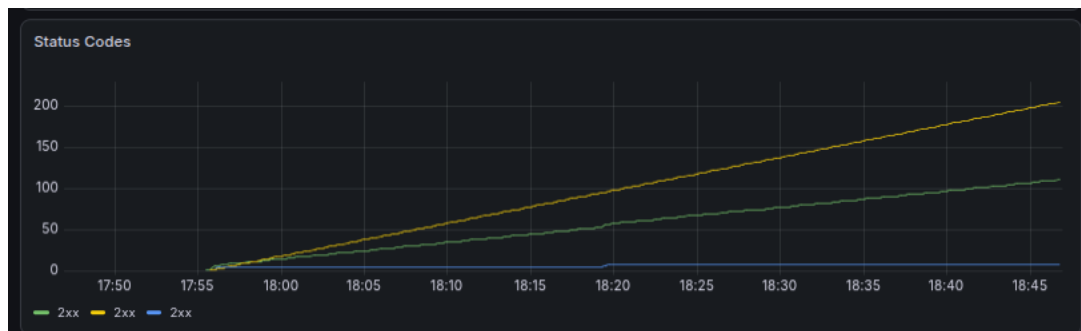
15.2 Dashboard de Grafana

Se configuró un dashboard con 4 paneles:

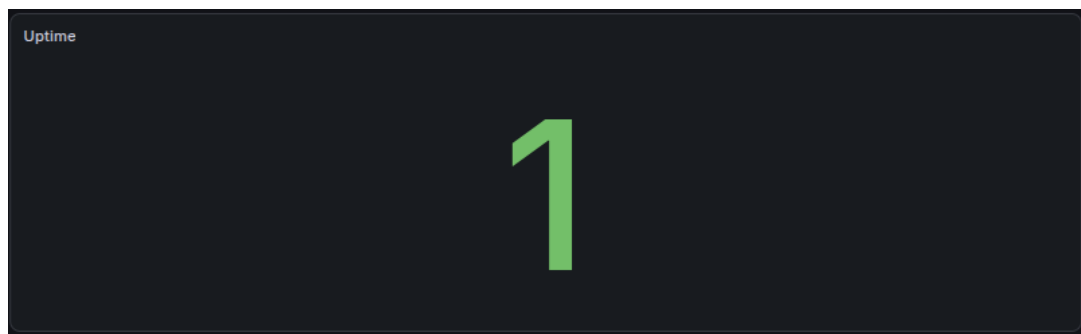
Panel 1 — Tráfico por endpoint: muestra el volumen acumulado de requests a /health, /metrics y /predict a lo largo del tiempo.



Panel 2 — Status codes: muestra la distribución de respuestas HTTP. Todos los responses son 2xx, confirmando que la API responde correctamente.



Panel 3 — Uptime: muestra el tiempo de actividad del servicio desde el último reinicio.



Panel 4 — Latencia del /predict: muestra el tiempo promedio de inferencia del endpoint de predicción. La latencia observada es ~0.015 segundos, lo cual es excelente para inferencia en tiempo real.



15.3 Dashboard de Grafana como código

Dashboard de Grafana incluido como código en el repositorio mediante provisioning automático — cualquier usuario que levante Docker ve el dashboard sin configuración adicional.



16. Monitoreo de Calidad de Datos con Evidently

16.1 ¿Qué es el drift de datos?

El drift ocurre cuando los datos que llegan en producción son estadísticamente diferentes a los datos con los que se entrenó el modelo. Si el perfil de los clientes cambia significativamente, el modelo puede empezar a predecir mal sin que haya ningún error técnico visible.

16.2 Metodología

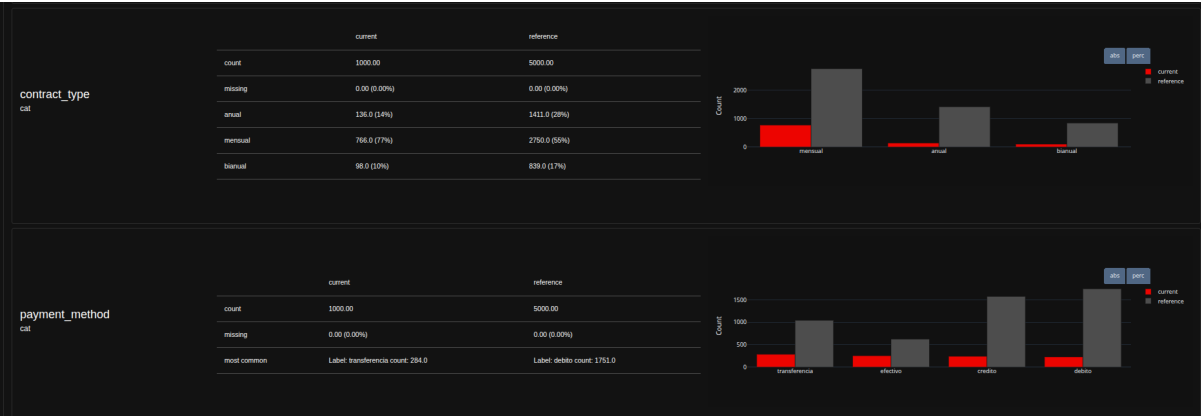
Se comparó el dataset de entrenamiento (5.000 registros, dataset de referencia) contra datos de producción simulados (1.000 registros) con un perfil de cliente diferente: clientes más nuevos, con cargos más altos y más tickets de soporte.

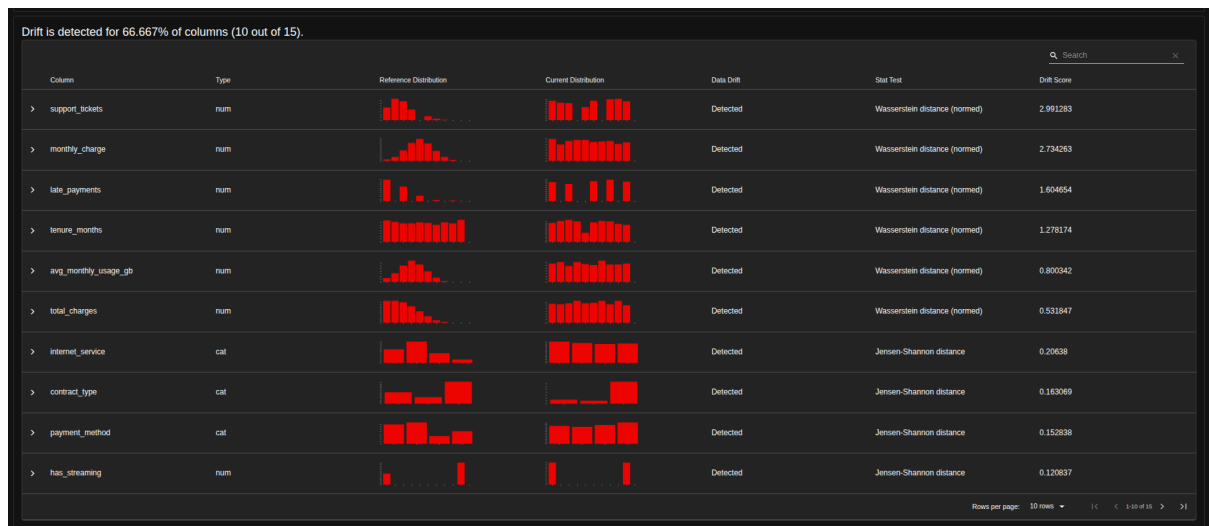
16.3 Resultados

Evidently detectó drift en **10 de 15 columns (66.7%)**.

Variable	Tipo	Drift Score	Método
support_tickets	numérica	2.991	Wasserstein distance
monthly_charge	numérica	2.734	Wasserstein distance
late_payments	numérica	1.605	Wasserstein distance
tenure_months	numérica	1.278	Wasserstein distance
avg_monthly_usage_gb	numérica	0.800	Wasserstein distance
total_charges	numérica	0.532	Wasserstein distance
internet_service	categorica	0.206	Jensen-Shannon distance
contract_type	categorica	0.163	Jensen-Shannon distance
payment_method	categorica	0.153	Jensen-Shannon distance
has_streaming	numérica	0.121	Jensen-Shannon distance

Dataset Summary		
Metric	Current	Reference
id column	None	None
target column	None	None
prediction column	None	None
date column	None	None
number of columns	15	15
number of rows	1000	5000
missing values	0	0
categorical columns	4	4
numeric columns	11	11
text columns	0	0
datetime columns	0	0
empty columns	0	0
constant columns	0	0
almost constant features	0	0
duplicated columns	0	0
almost duplicated features	0	0





16.4 Interpretación del negocio

Las variables con mayor drift (support_tickets, monthly_charge, tenure_months) son también las más correlacionadas con el churn según el análisis exploratorio (sección 3). Esto indica que los clientes en producción tienen un perfil de mayor riesgo que los del entrenamiento, lo que podría causar que el modelo subestime la probabilidad de churn real.

Acción recomendada: reentrenar el modelo incorporando los datos de producción para recalibrar la distribución esperada.



17. Suite de Tests Ampliada

Se expandió la suite de tests de 7 a 11 casos, incorporando las mejoras técnicas de la Fase 3:

Test	Descripción	Resultado
test_root	GET / devuelve 200	PASSED
test_health_check	GET /health valida modelo cargado	PASSED
test_prediccion_cliente_valido	POST /predict con datos válidos	PASSED
test_prediccion_payload_incompleto	Payload sin campos devuelve 422	PASSED
test_prediccion_tipo_incorrecto	Tipos incorrectos devuelven 422	PASSED
test_categoria_invalida_contract_type	"trimestral" devuelve 422	PASSED

Test	Descripción	Resultado
test_categoria_invalida_internet_service	"satelital" devuelve 422	PASSED
test_formato_respuesta	Campos y tipos correctos en response	PASSED
test_prediccion_cliente_bajo_riesgo	Cliente fiel tiene prob < 0.5	PASSED
test_prediccion_cliente_alto_riesgo	Cliente en riesgo tiene prob > 0.5	PASSED
test_payload_vacio	Payload vacío devuelve 422	PASSED

```
(venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ pytest tests/ -v --html=reports/test_report.html --self-contained-html
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0 -- /home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos/venv/bin/python3.12
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.11-76070011-generic-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.0.3', 'pluggy': '1.6.0'}, 'Plugins': {'Faker': '40.28
.1', 'metadata': '3.1.1', 'anyio': '4.13.0', 'html': '4.2.0', 'hydra-core': '1.3.2'}}
rootdir: /home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos
plugins: Faker-40.28.1, metadata-3.1.1, anyio-4.13.0, html-4.2.0, hydra-core-1.3.2
collected 11 items

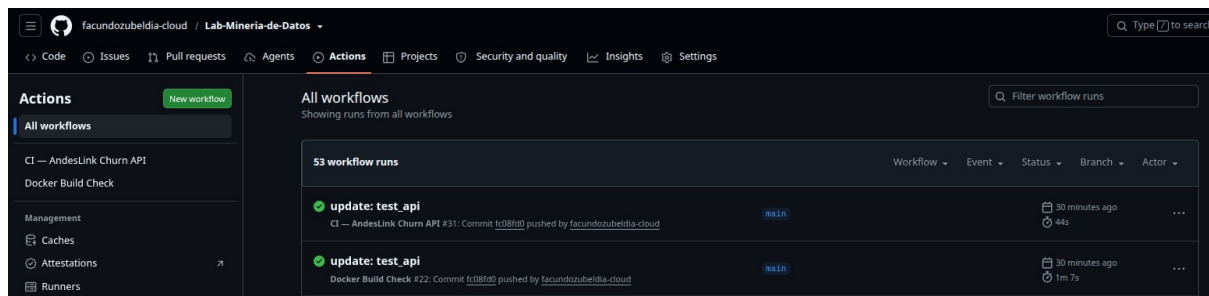
tests/test_api.py::test root PASSED [ 9%]
tests/test_api.py::test health check PASSED [ 18%]
tests/test_api.py::test prediccion cliente valido PASSED [ 27%]
tests/test_api.py::test prediccion payload incompleto PASSED [ 36%]
tests/test_api.py::test prediccion tipo incorrecto PASSED [ 45%]
tests/test_api.py::test categoria_invalida_contract type PASSED [ 54%]
tests/test_api.py::test categoria_invalida_internet service PASSED [ 63%]
tests/test_api.py::test formato respuesta PASSED [ 72%]
tests/test_api.py::test prediccion cliente bajo riesgo PASSED [ 81%]
tests/test_api.py::test prediccion cliente alto riesgo PASSED [ 90%]
tests/test_api.py::test payload vacio PASSED [100%]

----- Generated html report: file:///home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos/reports/test_report.html -----
11 passed in 1.64s
(venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$
```

18. GitHub Actions — CI/CD

Se mantienen dos workflows automáticos que se ejecutan con cada git push:

- **ci.yml** — corre los 11 tests con pytest. Si alguno falla, GitHub lo marca como fallido y bloquea el merge.
- **docker-build.yml** — construye ambas imágenes Docker y verifica que la API arranca correctamente. Detecta si alguien olvida agregar una dependencia al requirements-docker.txt.



19. Conclusiones Finales

Se completó el ciclo completo de MLOps para AndesLink:

- **Fase 1:** EDA, preparación de datos, comparación de modelos, serialización, MLflow y DVC.
- **Fase 2:** API de inferencia, dashboard Streamlit, Docker Compose, tests automatizados.
- **Fase 3:** Prometheus, Grafana, Evidently, mejoras técnicas de la API.

El sistema cumple con todos los requisitos de observabilidad: monitoreo técnico del servicio (latencia, tráfico, errores) y monitoreo de calidad de datos (drift). La detección de drift en 66.7% de las variables confirma que el sistema es capaz de identificar cuándo el modelo necesita ser reentrenado.

Limitaciones y trabajo futuro

- El dataset es sintético y los datos de producción son simulados; en producción real se registrarían los requests reales al endpoint /predict.
- Sin autenticación en la API; en producción se implementaría OAuth2 o API keys.
- El reentrenamiento es manual; en un sistema productivo se automatizaría con Airflow o similar cuando el drift supere un umbral definido.
- El remote de DVC es local; en producción apuntaría a S3 o GCS.

Desafíos técnicos y resolución de problemas

Durante el desarrollo del proyecto se enfrentaron y resolvieron las siguientes complicaciones críticas:

1. **Gestión de entornos (dependency hell):** al combinar librerías de entrenamiento (MLflow, Evidently) con las de despliegue (FastAPI, Streamlit), surgieron conflictos de versiones. **Resolución:** se implementaron archivos de requerimientos separados (requirements.txt para desarrollo y requirements-docker.txt para producción), asegurando imágenes Docker ligeras y estables.
2. **Conflictos de puertos:** en entornos locales, el puerto 9090 de Prometheus frecuentemente entraba en conflicto con servicios locales. **Resolución:** se implementó mapeo de puertos dinámico en docker-compose.yml.
3. **Persistencia en Grafana:** los dashboards configurados manualmente se perdían al reiniciar Docker. **Resolución:** se implementó el aprovisionamiento automatizado mediante archivos JSON y configuración YAML de Grafana para que el dashboard sea "código" y se levante automáticamente.
4. **Evolución de APIs:** al migrar entre versiones de Evidently (0.6.7), se encontraron cambios en la sintaxis de Report. **Resolución:** se ajustó el flujo de ejecución del objeto Report a la versión actual de la librería, asegurando la compatibilidad.